

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерной техники

Дисциплина: Системы искусственного интеллекта

Модуль 1. Лабораторная работа №1

Выполнил студент:

Немыкин Ярослав Алексеевич

Группа Р3322

Преподаватель:

Королёва Юлия Александровна

г. Санкт-Петербург, 2026

Table of contents (Оглавление)

АННОТАЦИЯ.....	3
ВВЕДЕНИЕ.....	4
БАЗА ЗНАНИЙ В PROLOG (ЛР1, ЧАСТЬ 1).....	5
Выбранный домен и словарь сущностей.....	5
Состав базы знаний.....	5
Структура исходного файла.....	6
Реализованные правила.....	7
Тестовые запросы и ответы интерпретатора.....	8
ОНТОЛОГИЯ В PROTÉGÉ (ЛР1, ЧАСТЬ 2).....	10
Перевод PROLOG → OWL.....	10
Классы и иерархия.....	10
ОбъектPROPERTY и DATAPROPERTY.....	11
Кардинальности и функциональность.....	11
Несовместимости и ограничения.....	12
Проверка REASONER и ограничения среды.....	12
Компетентностные вопросы и ожидаемые ответы.....	13
ЗАКЛЮЧЕНИЕ.....	15
ПРИЛОЖЕНИЕ.....	16

Аннотация

В рамках модуля разработана небольшая предметно-ориентированная база знаний для игровой среды с игроками, персонажами, классами, расами, противниками и местами рождения. На первом этапе знания представлены в виде фактов и правил на языке Prolog, что позволяет получать новые сведения посредством логического вывода. На втором этапе основные сущности и отношения перенесены в OWL-онтологию, созданную в Protégé. В онтологии заданы классы, индивиды, объектные и дататип-свойства, а также ограничения функциональности некоторых отношений. Получившаяся модель может рассматриваться как основа для знанийого компонента системы поддержки принятия решений (DSS), где Prolog отвечает за правилый вывод, а OWL — за структурированное представление предметной области. При этом плагины для Protégé в работе не подключались, поэтому полноценная работа с SPARQL и SWRL не выполнялась: в Protégé сформирована преимущественно структурная схема онтологии и набор индивидов.

Введение

Цель модуля — освоить базовые приёмы логического моделирования знаний: выделение сущностей предметной области, формализацию фактов и отношений, построение правил вывода, а также перенос полученной модели в онтологическое представление.

В работе рассматриваются два взаимодополняющих способа представления знаний. Prolog позволяет компактно описывать факты и правила и получать ответы на запросы за счёт унификации и логического вывода. OWL в Protégé используется для фиксации классов, индивидов, свойств и ограничений, благодаря чему структура предметной области становится явной и пригодной для дальнейшего машинного анализа.

Практическая ценность базы знаний и онтологии состоит в повторном использовании формализованных знаний, возможности проверять выводимые утверждения и сохранять прозрачность структуры модели. Связка БЗ → онтология → DSS рассматривается как последовательность подготовки знаний для системы поддержки решений: факты задают исходные данные, правила и ограничения задают логику, а онтология предоставляет формальную схему предметной области.

База знаний в Prolog (ЛР1, Часть 1)

Выбранный домен и словарь сущностей

В качестве предметной области выбрана игровая среда. В исходном Prolog-файле выделены игроки, персонажи, противники, классы, расы и деревни. Дополнительные отношения описывают выбор персонажа игроком, класс персонажа, расу, место и год рождения, а также враждебность между расами.

Сущность / отношение	Арность	Назначение
player	1	игрок
char	1	персонаж
enemy	1	противник
class	1	игровой класс
magical_class	1	магический класс
race	1	раса
village	1	деревня
playing	2	игрок принимает роль персонажа
char_info	2	персонаж относится к классу
hostile_race	2	враждебность между расами
born_race	2	раса персонажа
birth_info	3	год и место рождения

Таблица 1 – словарь сущностей

Состав базы знаний

Файл first.pl содержит 23 факта с одним аргументом и 22 факта с двумя или более аргументами. В базе присутствуют четыре игрока, четыре персонажа, два противника, четыре класса, два из которых магические, четыре расы и три деревни. Отношения представлены привязками игроков к персонажам, классами, расами, враждебностью рас и сведениями о рождении.

В разделе правил реализованы семь правил: knows_history/2, can_inspire/1, will_conflict/2, can_tank/1, can_suppressed/2, party_tension/2 и age/3. Они используют композицию условий, конъюнкцию, дизъюнкцию, отрицание неравенства, а также арифметическое вычисление.

Структура исходного файла

- Факты об игроках, персонажах и противниках.
- Факты о классах и магических классах.
- Факты о расах и враждебности рас.
- Связи персонажей с расами и сведения о рождении.
- Правила логического вывода.

Фрагмент исходной базы знаний:

```
% Ассистент в ведении партий нри
% Игроки
player(ivan).
player(oleg).
player(artem).
player(vanya).

% Персонажи
char(torvald).
char(illidan).
char(shmig).
char(senshi).
% злодеи
enemy(buga).
enemy(vuga).

% Привязки персонажей к игрокам
playing(ivan, torvald).
playing(oleg, shmig).
playing(artem, illidan).
playing(vanya, senshi).

% Классы
class(rogue).
class(bard).
class(mage).
class(warrior).
% Магические классы
magical_class(mage).
magical_class(bard).
```

Рисунок 1 — Фрагмент фактов из *first.pl*.

Реализованные правила

```
% Воин может танковать
can_tank(Char) :-
    char_info(Char, warrior).

% Персонаж может эмоционально подавить другого, если разница в возрасте
% больше на 20 лет
can_suppressed(Char1, Char2) :-
    ( char(Char1); enemy(Char1)),
    ( char(Char2); enemy(Char2)),
    Char1 \= Char2,
    birth_info(Char1, Year1, _),
    birth_info(Char2, Year2, _),
    Difference is Year2 - Year1,
    Difference > 20.

% Напряжение между игроками есть, если их персонажи конфликтуют
party_tension(Player1, Player2) :-
    playing(Player1, Char1),
    playing(Player2, Char2),
    Player1 \= Player2,
    will_conflict(Char1, Char2).

% Сколько лет персонажу сейчас
age(Char, Age, Now) :-
    birth_info(Char, Year, _),
    Age is Now - Year.
```

Рисунок 2 — Примеры правил.

Наиболее содержательным является правило `will_conflict/2`. Оно объединяет несколько альтернатив: конфликт возможен из-за враждебности рас либо из-за участия противника в игровой связи. В условии используются дизъюнкции (`;`), конъюнкции через последовательность целей и ограничение `Char1 \= Char2`.

```
% Персонажи будут конфликтовать, если враждебной рассы друг к другу или
% враги
will_conflict(Char1, Char2) :-
    ( char(Char1); enemy(Char1)),
    ( char(Char2); enemy(Char2)),
    Char1 \= Char2,
    born_race(Char1, Race1),
    born_race(Char2, Race2),
    ( hostile_race(Race1, Race2);
      enemy(Char1), playing(_, Char2));
      enemy(Char2), playing(_, Char1)
    ).
```

Рисунок 3 — Правила конфликтов.

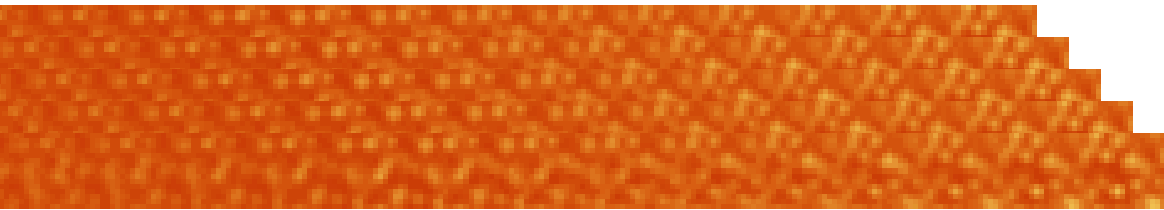
Тестовые запросы и ответы интерпретатора

Тип	Запрос	Ответ / интерпретация
Простой поиск факта	?- player(ivan).	true.
Поиск персонажа	?- char(torvald).	true.
Конъюнкция	?- playing(P, C), char_info(C, mage).	P = artem, C = illidan.
Дизъюнкция	?- can_tank(C) ; can_inspire(C).	C = senshi ; C = torvald ; C = illidan ; C = vuga.
Отрицание	?- can_suppressed(C1, C2), \+ will_conflict(C1, C2).	C1 = illidan, C2 = torvald ; C1 = illidan, C2 = shmig ;
		C1 = senshi, C2 = torvald
Правило истории	?- will_conflict(torvald, Who).	Who = shmig ; Who = buga ; Who = vuga
Правило напряжённости	?- party_tension(P1, P2).	P1 = ivan, P2 = oleg ; P1 = oleg, P2 = ivan ; P1 = oleg, P2 = vanya ; P1 = artem, P2 = vanya ; P1 = vanya, P2 = oleg ; P1 = vanya,

		P2 = artem
Арифметическое правило	?- age(illidan, Age, 2024).	Age = 1024.
Возрастное условие	?- can_suppressed(shmig, torvald).	true.

Таблица 2 – запросы и ответы от интерпретатора

Приведённые ответы отражают логику исходного файла. Для запросов, где Prolog может возвращать несколько решений, фактический порядок вариантов определяется порядком фактов и ветвлений в программе.



Онтология в Protégé (JP1, Часть 2)

Перевод Prolog → OWL

В файле first-ontology.rdf исходные понятия представлены средствами OWL/RDF. Выделены классы Character, Enemy, Player, Race, Village, Class и MagicClass. Класс Enemy задан как подкласс Character, а MagicClass — как подкласс Class. Индивиды соответствуют персонажам, игрокам, расам, деревням и классам из Prolog. Сам файл был создан при помощи интерфейса в программе Protégé

Prolog	OWL элемент	Комментарий
char(X)	Character	Класс персонажей.
enemy(X)	Enemy $\sqsubseteq$ Character	Противник моделируется как подкласс персонажа.
player(X)	Player	Класс игроков.
race(X)	Race	Класс рас.
village(X)	Village	Класс деревень.
playing(P,C)	playing (ObjectProperty)	Player $\rightarrow$ Character.
char_info(C,Class)	hasClass (ObjectProperty)	Character $\rightarrow$ Class.
born_race(C,R)	bornRace (ObjectProperty)	Character $\rightarrow$ Race.
birth_info(C,Year,V)	birthYear + birthPlace	Разделение тернарного факта на два свойства.
hostile_race(R1,R2)	hostileRace (ObjectProperty)	Race $\rightarrow$ Race.

Таблица 3 — Мини-матрица трассируемости Prolog $\leftrightarrow$ OWL.

Классы и иерархия

В текущем RDF-файле зафиксирована компактная иерархия. Enemy является подклассом Character, а MagicClass — подклассом Class. Остальные классы представлены на одном уровне. Это соответствует схеме предметной области, реализованной в исходной версии онтологии.

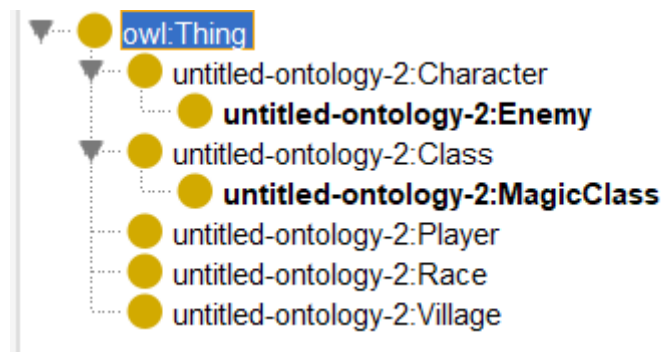


Рисунок 4 — Иерархия классов.

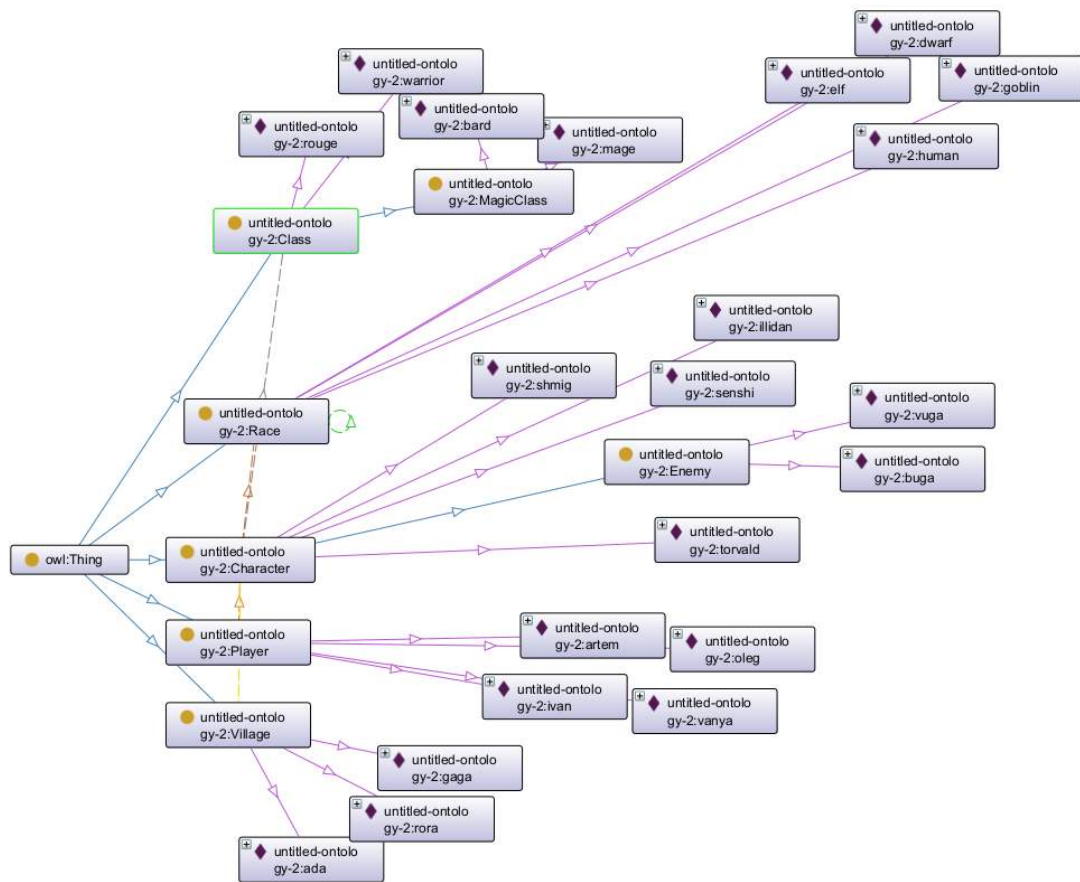


Рисунок 5 — Граф онтологии.

ObjectProperty и DataProperty

В онтологии определены следующие объектные свойства: `birthPlace`, `bornRace`, `hasClass`, `hostileRace` и `playing`. Для них заданы домены и диапазоны. Например, `birthPlace` имеет домен `Character` и диапазон `Village`, `bornRace` — `Character` и `Race`, `hasClass` — `Character` и `Class`, `playing` — `Player` и `Character`.

Дататип-свойство `birthYear` имеет домен `Character` и диапазон `xsd:integer`. Тем самым год рождения представлен числовым значением, а не отдельным объектом.

Кардинальности и функциональность

В работе несколько объектных свойств объявлены функциональными (FunctionalProperty): birthPlace, bornRace, hasClass и hostileRace. Свойство playing дополнительно объявлено InverseFunctionalProperty. Это означает, что модель ограничивает число значений соответствующих отношений и задаёт семантику уникальности. Ограничения выражены через функциональность свойств.

Несовместимости и ограничения

В работе явные конструкции owl:disjointWith и числовое ограничение вида $\text{Age} \geq 0$ отсутствуют. Так как подобных полей не создавалось при проектировании.

Проверка reasoner и ограничения среды

При использовании HermiT проблемы выявлены не были.

Плагины и дополнительные расширения Protégé не удалось подключить. Поэтому полноценное выполнение SPARQL-запросов и SWRL-правил в рамках работы не удалось. Практическая часть в Protégé ограничилась построением онтологической схемы, описанием свойств и размещением индивидов. Однако, удалось составить DL Query запросы.

Query (class expression)

untitled-ontology-2:Character and (untitled-ontology-2:hasClass value untitled-ontology-2:mage or untitled-ontology-2:hasClass value untitled-ontology-2:bard)

Execute

Add to ontology

Query results

Instances (3 of 3)

untitled-ontology-2:illidan

untitled-ontology-2:torvald

untitled-ontology-2:vuga

1

Query (class expression)

untitled-ontology-2:Character and (untitled-ontology-2:hasClass value untitled-ontology-2:warrior)

Execute

Add to ontology

Query results

Instances (2 of 2)

untitled-ontology-2:buga

untitled-ontology-2:senshi

Query (class expression)

untitled-ontology-2:Enemy

Execute

Add to ontology

Query results

Instances (2 of 2)

untitled-ontology-2:buga

untitled-ontology-2:vuga

Рисунок 6 – работа DL Query

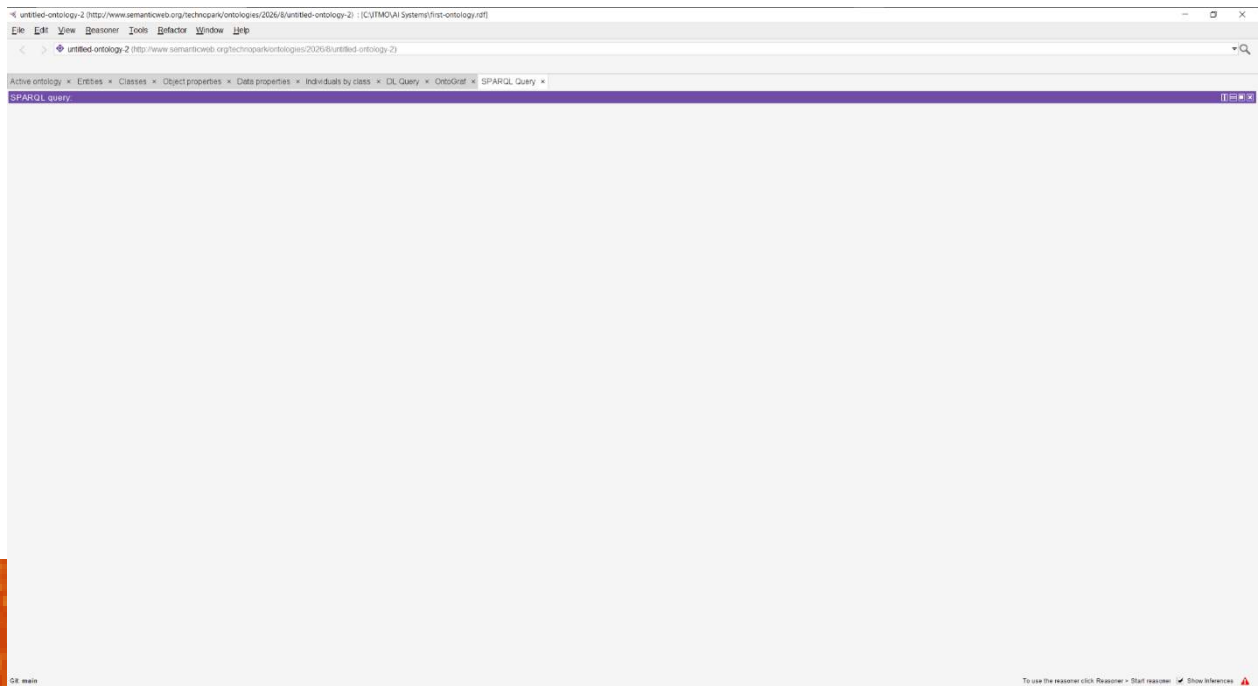


Рисунок 7 – ошибки SPARQL.

```

Plugin: OWLAPI (3.0.0)
Plugin: OWLAPI RDF Library (3.0.1)
Plugin: SPARQL Query Plugin (3.0.0)
Plugin: SWRLTab Plugin (2.1.3) was not successfully started. Please see the Protégé log for more details.

Creating and setting up empty (default) editor kit
OWL API Version: 4.5.29.2024-05-13T12:11:03Z
Cannot generate ontology catalog for ontology at http://www.semanticweb.org/technopark/ontologies/2026/8/untitled-ontology-6. URI scheme is not "file"
Error whilst adding menu item: {}
java.lang.IllegalAccessException: class org.protege.editor.core.ui.workspace.Workspace cannot access class com.sun.java.swing.plaf.windows.WindowsLookAndFeel (in module java.desktop) because module java.desktop does not export com.sun.java.swing.plaf.windows to unnamed module @2bb1b0e9
    at java.base/jdk.internal.reflect.Reflection.newIllegalAccessException(Reflection.java:392)
    at java.base/java.lang.reflect.AccessibleObject.checkAccess(AccessibleObject.java:674)
    at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:490)
    at java.base/java.lang.reflect.ReflectAccess.newInstance(ReflectAccess.java:128)
    at java.base/jdk.internal.reflect.ReflectionFactory.newInstance(ReflectionFactory.java:347)
    at java.base/java.lang.Class.newInstance(Class.java:647)
    at org.protege.editor.core.ui.workspace.Workspace.addLookAndFeelMenuItem(Workspace.java:219)
    at org.protege.editor.core.ui.workspace.Workspace.installLookAndFeelMenu(Workspace.java:195)
    at org.protege.editor.core.ui.workspace.Workspace.initialiseExtraMenuItems(Workspace.java:128)

```

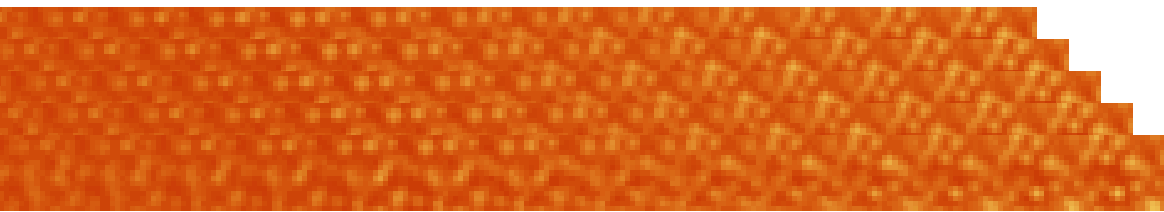
Рисунок 8 – ошибки SWRL.

Компетентностные вопросы и ожидаемые ответы

Поскольку SPARQL и SWRL в текущей среде не использовались, ниже приведены не результаты выполнения запросов, а формулировки компетентностных вопросов, которые проверяют содержание созданной модели. Они могут быть реализованы в Protégé при подключении соответствующих средств.

Компетентностный вопрос	Ожидаемый ответ по данным RDF
Какие игроки представлены в онтологии?	ivan, oleg, artem, vanya.
Какими персонажами управляют игроки?	ivan→torvald, oleg→shmig, artem→illidan, vanya→senshi.
Какой класс имеет illidan?	mage.

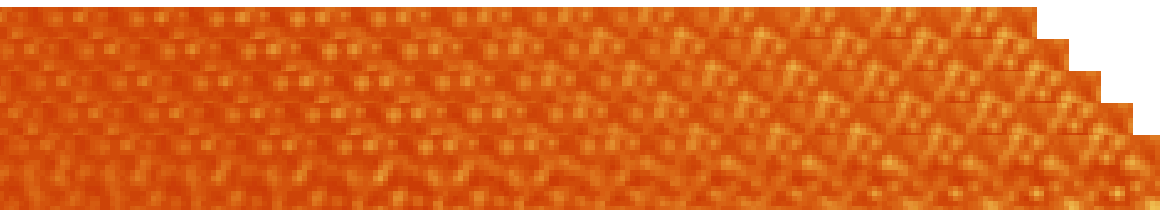
Какие персонажи родились в деревне ada?	torvald, shmig, senshi.
Какие расы связаны отношением hostileRace?	goblin↔dwarf, goblin↔human, elf↔dwarf.



Заключение

В ходе модуля была разработана база знаний по игровой предметной области на Prolog и создана соответствующая OWL -модель в Protégé. Prolog-часть содержит достаточный набор одноместных и многоместных фактов, а также семь правил, использующих конъюнкцию, дизъюнкцию, отрицание неравенства и арифметический расчёт. Это позволяет демонстрировать как прямой поиск фактов, так и получение новых сведений через правила.

В онтологии выделены основные классы, индивиды, объектные и дататип-свойства, а часть отношений сделана функциональной. Плагины Protégé подключить не удалось, поэтому SPARQL и SWRL не были использованы.



Приложение

Ссылки на исходные файлы, упоминаемые в отчете: [Гитхаб репозиторий](#)

